

For this assignment, we developed a Python-based operating system performance simulator designed to demonstrate disk management, CPU monitoring, memory management, threading, and fault handling. The program retrieves real-time system information using Python's psutil library and displays several operating system metrics that can be used to evaluate overall system performance.

One of the optimization techniques implemented in the simulator was fault handling through exception management. The program intentionally generates a divide-by-zero exception and handles the error using a try-except block. Instead of causing the application to terminate unexpectedly, the exception is caught and displayed to the user. This demonstrates how operating systems and applications can continue functioning while properly handling failure states. Proper error handling improves reliability by preventing unexpected crashes and allowing systems to recover gracefully from common faults.

Several operating system performance metrics were used throughout the application. CPU utilization and logical CPU count were collected to provide information about processor usage and available processing resources. Memory metrics included total memory, used memory, and virtual memory statistics. Disk metrics included total disk space, free disk space, and file size information. These metrics are important because they help administrators monitor resource consumption, identify bottlenecks, and evaluate whether a system is operating efficiently under current workloads.

For memory management, the simulator utilizes information provided by the operating system regarding both physical and virtual memory usage. Virtual memory is an important optimization technique because it allows the operating system to extend available memory resources beyond physical RAM. Monitoring memory usage helps maintain system stability by identifying situations where memory resources may become constrained and affect performance.

Scheduling concepts were demonstrated through the use of multiple threads. Two independent threads were created and executed concurrently while displaying their execution information. Threading serves as a scheduling optimization because multiple tasks can be processed simultaneously instead of waiting for each task to complete sequentially. Modern operating systems rely on thread scheduling to improve responsiveness and distribute workloads across available CPU resources.

Resource allocation optimization was demonstrated by independently monitoring CPU, memory, and disk resources while allowing concurrent execution through threading. This reflects how operating systems distribute resources among multiple processes while preventing any single task from consuming excessive resources. Efficient resource allocation improves overall system performance, responsiveness, and stability.

Overall, this simulator demonstrates several operating system optimization techniques including fault handling, memory management, thread scheduling, and resource allocation. These mechanisms contribute to improved reliability, efficiency, and performance while providing visibility into the system resources being utilized.